Transfer Learning

Redes Neuronales Profundas y Transferencia del Conocimiento

Prof. D.Sc. BARSEKH-ONJI Aboud

Facultad de Ingeniería
Universidad Anáhuac México

April 23, 2026

Course Outline

Agenda

The Challenge of Training Deep Networks

Training a CNN from scratch requires:

- **Millions of labeled images** — e.g., ImageNet: 1.2M images, 1000 classes
- **Days or weeks of computation** — even on high-end GPUs
- **Expert knowledge** for architecture design and hyperparameter tuning
- **Massive storage** for dataset management

Reality in applied projects:

Most real-world problems have **hundreds, not millions** of labeled examples. Training from scratch under these conditions leads to severe *overfitting*.

The Human Learning Analogy

How humans learn:

- A radiologist learns anatomy *before* interpreting X-rays
- A civil engineer applies physics *before* designing bridges
- General knowledge **transfers** to new specific tasks

How neural networks traditionally learned:

- Every new task $\rightarrow$ train from **zero**
- All knowledge reacquired from scratch
- Ignores previously learned representations

Key Insight

Transfer Learning bridges this gap: knowledge learned on one task is **reused** to accelerate learning on a new, related task.

Agenda

Definition: Transfer Learning

Transfer Learning is a machine learning technique in which a model trained on a **source task** (large dataset) is adapted to solve a **target task** (smaller dataset), by reusing the learned feature representations.

Source task:

- Large dataset (e.g., ImageNet)
- General features: edges, textures, shapes
- Pre-trained network already available

Target task:

- Small dataset (tens to hundreds of images)
- Specific domain (medical, industrial, commercial)
- Only final layers retrained

Why It Works: Feature Hierarchy in CNNs

Deep CNNs learn **hierarchical representations**:

- **Early layers** — generic features:
  - Edges, gradients, color blobs
  - Useful for *any* visual task
- **Middle layers** — semi-specific features:
  - Textures, patterns, object parts
  - Transferable to related domains
- **Final layers** — task-specific:
  - Class scores for the original 1000 classes
  - **Must be replaced** for a new task

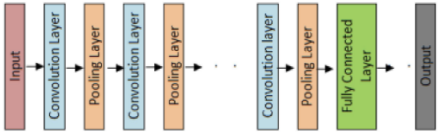


Figure 1: Feature hierarchy in a deep CNN

When to Use Transfer Learning - Recommended scenarios:

Situation	Approach	Result
Small dataset, similar domain	Transfer + fine-tune last layers	Excellent
Small dataset, different domain	Transfer + retrain more layers	Good
Large dataset, similar domain	Fine-tune entire network	Very good
Large dataset, different domain	Train from scratch	Possible

Most common case in practice:

Small dataset + domain related to ImageNet (objects, scenes, textures) $\Rightarrow$ **Transfer Learning is the go-to approach.**

Agenda

Two Main Strategies

1. Feature Extraction

- **Freeze** all convolutional layers
- Only train the new classification head
- Fast, less data needed
- Risk: limited adaptation to new domain

2. Fine-Tuning

- **Unfreeze** some or all layers
- Retrain with a **low learning rate**
- More flexible, better accuracy
- Risk: catastrophic forgetting if LR too high

Strategy used in Ex5:

Fine-tuning of the last convolutional layer (`conv10`) with high learning rate factors, while keeping all other layers at the pre-trained weights.

Transfer Learning vs. Training from Scratch

Training from Scratch

- Millions of samples required
- Days/weeks of training
- High risk of underfitting
- Justified only for very unique domains

Transfer Learning

- Hundreds of samples sufficient
- Minutes to hours of training
- High accuracy from the start
- Practical for most real problems

Transfer Learning has become the *de facto* standard approach for applied deep learning with limited data.

Learning Rate Strategy in Fine-Tuning

The key to successful fine-tuning: **differential learning rates**.

Layer-wise Learning Rate Control

- **Pre-trained layers**: very low LR (e.g., $\eta = 0.0001$)
  $\rightarrow$ Preserves general features already learned
- **New classification layers**: high LR factor (e.g., $\times 10$)
  $\rightarrow$ Adapts quickly to the new classes

Catastrophic Forgetting

Using a high global learning rate will **destroy** the pre-trained weights. The network will "forget" what it learned on ImageNet, losing the advantage of transfer learning entirely.

Agenda

Available Pre-trained Networks

MATLAB's Deep Learning Toolbox includes a curated gallery of pre-trained CNNs:

Network	**Parameters**	**Input Size**	**Strength**
SqueezeNet	1.24 M	227×227	Compact, fast
GoogLeNet	7 M	224×224	Balanced
ResNet-50	25 M	224×224	High accuracy
ResNet-101	44 M	224×224	Very deep
VGG-16	138 M	224×224	Simple, powerful
EfficientNet-b0	5.3 M	224×224	Efficient scaling

SqueezeNet Architecture

SqueezeNet (Iandola et al., 2016):

- Designed for **extreme compactness**
- Only **1.24 M parameters** (vs. 60 M in AlexNet)
- Comparable accuracy on ImageNet
- Key innovation: **Fire modules**
  - *Squeeze*: 1×1 convolutions (dimensionality reduction)
  - *Expand*: $1 \times 1 + 3 \times 3$ convolutions
- Final layer: `conv10` with 1000 filters
- Input size: $227 \times 227 \times 3$

Why SqueezeNet for teaching?

- No additional support package
- Trains on CPU in minutes
- Clear, inspectable architecture
- Ideal for small datasets

SqueezeNet — Fire Module

Fire Module Structure

Each Fire module consists of two stages: **Squeeze** (1×1 conv) $\longrightarrow$ **Expand** (1×1 + 3×3 conv, concatenated)

Squeeze layer

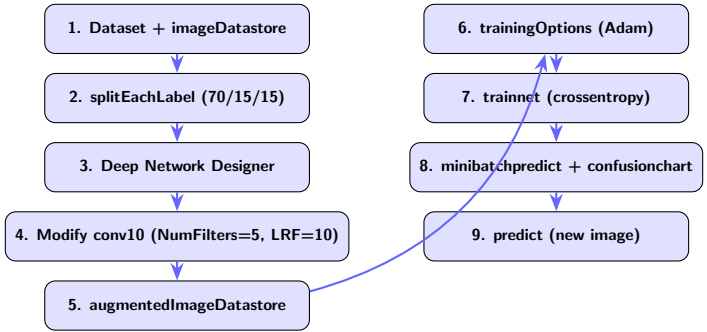
- Only 1×1 convolution filters
- Reduces the number of input channels (dimensionality compression)
- $9\times$ fewer parameters than a 3×3 filter

Expand layer

- Mix of 1×1 and 3×3 convolution filters
- Outputs concatenated along the channel axis
- Recovers spatial information after squeeze

Agenda

Complete Pipeline Overview



The Adam Optimizer

Adam (*Adaptive Moment Estimation*) maintains running estimates of first and second moments of the gradient:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{1}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{2}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{3}$$

- $\hat{m}_t$, $\hat{v}_t$: bias-corrected estimates
- **Adaptive**: each parameter has its own effective LR
- **Robust** for small, heterogeneous datasets
- Default: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Training Options

```
%% 4. OPCIONES DE ENTRENAMIENTO
options = trainingOptions("adam", ...
    InitialLearnRate=0.0001, ...          % Bajo: protege pesos preentrenados
    MaxEpochs=8, ...                      % Pocas epocas (TL converge rapido)
    ValidationData=imdsValidation, ...    % Monitor de sobreajuste
    ValidationFrequency=5, ...            % Valida cada 5 iteraciones
    MiniBatchSize=11, ...                 % 52 imgs / 11 ~ 5 iter/epoca
    Plots="training-progress", ...        % Grafica perdida y precision
    Metrics="accuracy", ...
    Verbose=false);
```

Training Options

Option	Value	Justification
`InitialLearnRate`	0.0001	Protects pre-trained weights
`MaxEpochs`	8	Fast convergence in TL
`MiniBatchSize`	11	Even split of 52 training imgs

Cross-Entropy Loss Function

`trainnet` uses **categorical cross-entropy** as the loss function:

$$\mathcal{L} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i) \tag{4}$$

- C: number of classes (5 in our example)
- y_i: true label (one-hot encoded: 1 for correct class, 0 otherwise)
- $\hat{y}_i$: predicted probability for class i (output of softmax)
- Penalizes the network more when it is **confident and wrong**

Agenda

Hyperparameter Sensitivity Analysis

Change	Effect	Why
↑ `MaxEpochs` (e.g., 20)	Overfitting	Memorizes training data
↑ `InitialLearnRate` (0.01)	Catastrophic forgetting	Destroys pre-trained weights
↓ `WeightLearnRateFactor` (1)	Slower adaptation	`conv10` learns too slowly
Remove augmentation	More overfitting	Fewer effective training samples
Use ResNet-50 instead	Higher accuracy	More parameters, richer feature

Try with MATLAB Experiment Manager:

The **Experiment Manager** app runs multiple hyperparameter configurations in parallel and plots comparison curves automatically.

Questions for Reflection

1. What happens if you increase `MaxEpochs` to 20? How does the training/validation gap evolve?
2. What happens if `InitialLearnRate` is set to 0.01? What does this tell us about catastrophic forgetting?
3. What would be the effect of setting `WeightLearnRateFactor = 1` in `conv10`?
4. Which pre-trained network would give better results on this dataset? Justify your answer in terms of parameters, depth, and input size.
5. How does the confusion matrix change with and without data augmentation? What does this tell us about the role of regularization in Transfer Learning?

Key Concepts — Reference Table

Concept	Meaning	In Ex5
Transfer Learning	Reuse weights from a source task	SqueezeNet (ImageNet) $\rightarrow$ 5 classes
Fine-tuning	Selective retraining of final layers	Only `conv10` modified aggressively
`imageDatastore`	Efficient image container (batch I/O)	Reads 75 images without RAM overload
Data Augmentation	Artificial training variants	Flip + $\pm$30px translations
Adam	Adaptive gradient optimizer	Converges well on small datasets
Cross-entropy loss	Classification loss function	Measures predicted vs. true distribution
Confusion matrix	Per-class error visualization	Identifies class-specific confusions
`minibatchpredict`	Batch prediction (efficient)	Classifies test set in batches
`scores2label`	Scores $\rightarrow$ class labels (argmax)	Converts probabilities to class names

Agenda

Summary

What we covered:

- Why training from scratch is impractical for small datasets
- How CNNs learn hierarchical features that generalize
- Feature Extraction vs. Fine-Tuning strategies
- Pre-trained networks available in MATLAB
- SqueezeNet architecture and Fire modules
- Full pipeline: data $\rightarrow$ train $\rightarrow$

Key takeaways:

Transfer Learning in practice

- **Low LR** for pre-trained layers
- **High LR factor** for new classification head
- **Data augmentation** is essential with small datasets
- **Confusion matrix** $>$ global accuracy for diagnosis
- Deep Network Designer enables GUI-based TL without code

References and Further Reading

MathWorks Documentation

- https://la.mathworks.com/discovery/transfer-learning.html
- https://la.mathworks.com/help/deeplearning/ug/transfer-learning-with-deep-network-designer.html

Key Papers

- Iandola et al. (2016) — *SqueezeNet: AlexNet-level accuracy with 50x fewer parameters*
- Pan & Yang (2010) — *A Survey on Transfer Learning*, IEEE TKDE
- Yosinski et al. (2014) — *How transferable are features in deep neural networks?*
- Kornblith et al. (2019) — *Do Better ImageNet Models Transfer Better?*